

[CS230] Sequence Models: Vanishing Gradients & GRU

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-9. Deep Learning Fundamentals & CNNs - *Completed*

Chapter 10. Sequence Models (Current Part)

- 10.1 RNN Basics - *Completed*
- 10.2 Various RNN Architectures - *Completed*
- **10.3 Vanishing Gradients & Gated Units**
 - * The Problem: Vanishing Gradients
 - * The Solution: Gating Mechanism
 - * GRU (Gated Recurrent Unit)
 - * Implementation: Update & Reset Gates
- 10.4 LSTM (Long Short-Term Memory) - *Upcoming*

지난 시간 복습 및 연결

지난 시간에 배운 기본 RNN은 이론적으로는 완벽해 보입니다. 과거 정보를 현재로 전달하니까요. 하지만 실제로는 문장이 10단어만 넘어가도 앞부분 내용을 까맣게 잊어버립니다. **"The cat, which ate ..., was full."** RNN은 중간 수식어가 길어지면 주어 'cat(단수)'을 잊어버리고 동사 'was'를 예측하지 못합니다. 이것이 바로 **기울기 소실 (Vanishing Gradient)** 문제입니다. 오늘은 이 난제를 해결한 **GRU**를 배웁니다.

1 Unit Overview

□ 핵심 목표

이 단원은 RNN의 장기 의존성 문제를 해결하는 게이트(Gate) 메커니즘을 다룹니다.

- **원인:** 역전파 시 기울기가 지수적으로 작아져서 초기 기억이 사라지는 수학적 원리를 이해합니다.
- **해결:** 정보를 "얼마나 유지하고 버릴지" 결정하는 **게이트(Gate)** 개념을 도입합니다.
- **모델:** LSTM의 간소화 버전인 **GRU**의 구조와 수식을 마스터합니다.

2 Essential Terminology

용어	의미	역할
Vanishing Gradient	기울기 소실	깊은 신경망 학습을 방해하는 주범.
Gate (Γ)	문지기 (0 1)	정보 흐름을 제어하는 시그모이드 함수.
Update Gate (Γ_u)	업데이트 게이트	과거 기억을 얼마나 유지할지 결정.
Reset Gate (Γ_r)	리셋 게이트	과거 기억을 얼마나 무시할지 결정.

3 Core Concepts: 기억의 보존

3.1 1. The Problem: Vanishing Gradients

RNN에서 $t = 100$ 시점의 오차를 $t = 1$ 시점까지 전파하려면 가중치 행렬 W 를 100번 곱해야 합니다.

□ 복사본의 복사본 (직관적 비유)

문서를 복사하고, 그 복사본을 또 복사하는 과정을 100번 반복한다고 상상해 보세요. 조금이라도 흐릿해지면 ($W < 1$), 100번째 복사본은 백지가 되어버립니다. 반대로 진해지면 ($W > 1$), 검은색 잉크 덩어리가 됩니다(Exploding).

3.2 2. The Solution: Gated Recurrent Unit (GRU)

핵심 아이디어는 "기억을 위한 전용 금고(Memory Cell)를 만들고 문지기를 세우자"는 것입니다.

□ GRU의 핵심 수식 (핵심 수식)

$$c^{(t)} = \Gamma_u \cdot \tilde{c}^{(t)} + (1 - \Gamma_u) \cdot c^{(t-1)}$$

- Γ_u (Update Gate): 0이면 문을 닫습니다.
- $\Gamma_u \approx 0$ 일 때: $c^{(t)} \approx c^{(t-1)}$.
- 의미: 과거의 기억($c^{(t-1)}$)이 아무런 변형 없이(행렬 곱셈 없이) 그대로 복사되어 다음으로 넘어갑니다. 기울기 소실 없이 고속도로처럼 전달됩니다.

4 Deep Dive: GRU Gates Detail

GRU는 두 개의 게이트를 사용합니다.

1. **Update Gate (Γ_u):** "이 기억을 계속 가져갈까?" (단수/복수 정보 유지)

$$\Gamma_u = \sigma(W_u[c^{(t-1)}, x^{(t)}] + b_u)$$

2. **Reset Gate (Γ_r):** "이제 과거는 잊을까?" (문침표가 나오면 리셋)

$$\Gamma_r = \sigma(W_r[c^{(t-1)}, x^{(t)}] + b_r)$$

3. **Candidate Memory ($\tilde{c}$):** 새로운 기억 후보

$$\tilde{c}^{(t)} = \tanh(W_c[\Gamma_r \cdot c^{(t-1)}, x^{(t)}] + b_c)$$

5 Implementation: GRU Cell

```
1 import numpy as np
2
3 def sigmoid(x):
4     return 1 / (1 + np.exp(-x))
5
6 def gru_cell_forward(xt, a_prev, parameters):
7     """
8     xt: 현재 입력
9     a_prev: 과거 은닉 상태 (예전 GRU Memory Cell 와 c 동일)
10    """
11    # 파라미터 로드
12    W_u = parameters["Wu"] # Update Gate Weights
13    W_r = parameters["Wr"] # Reset Gate Weights
14    W_c = parameters["Wc"] # Candidate Memory Weights
15
16    # 입력 결합 (Concatenate)
17    concat = np.concatenate((a_prev, xt), axis=0)
18
19    # 1. Update Gate (Gamma_u)
20    Gamma_u = sigmoid(np.dot(W_u, concat) + parameters["bu"])
21
22    # 2. Reset Gate (Gamma_r)
23    Gamma_r = sigmoid(np.dot(W_r, concat) + parameters["br"])
24
25    # 3. Candidate Memory (c_tilde)
26    # 과거 기억에 Reset Gate 적용
27    concat_reset = np.concatenate((Gamma_r * a_prev, xt), axis=0)
28    c_candidate = np.tanh(np.dot(W_c, concat_reset) + parameters["bc"])
29
30    # 4. Final Memory Update (The Magic Formula)
31    # 가 u 이면 0 과거 기억 보존, 이면 1 새 기억으로 덮어씀
32    c_next = Gamma_u * c_candidate + (1 - Gamma_u) * a_prev
33
34    return c_next
```

Listing 1: GRU Cell Forward Implementation

6 FAQ & Pitfalls

Gradient Clipping (기울기 자르기) (오해 방지 가이드)

기울기 소실의 반대는 기울기 폭발(**Exploding Gradient**)입니다. 숫자가 너무 커져서 'NaN'이 뜹니다. 해결책은 간단합니다. 기울기 벡터의 크기(Norm)가 특정 값(예: 5)을 넘으면 강제로 줄여버리는 **Gradient Clipping**을 사용합니다.

Q. GRU와 LSTM 중 뭐가 더 좋나요?

A. 정답은 없습니다. GRU는 구조가 단순해서(게이트 2개) 연산이 빠르고 데이터가 적을 때 유리합니다. LSTM은 구조가 복잡하지만(게이트 3개) 표현력이 더 좋습니다. 보통 LSTM을 기본으로 쓰고, 속도가 중요하면 GRU를 씁니다.

다음 단계 (Next Step)

GRU는 훌륭하고 단순합니다. 하지만 때로는 더 섬세한 제어가 필요합니다. GRU보다 조금 더 복잡하지만, 더 강력하고 널리 쓰이는 형님이 있습니다.

다음 시간에는 GRU의 확장판이자 RNN의 사실상 표준(De facto Standard), **[LSTM (Long Short-Term Memory)]**에 대해 다루겠습니다. 3개의 게이트(Forget, Input, Output)가 어떻게 기억을 수술하듯 정교하게 다루는지 알아보겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Vanishing Gradient:** 시퀀스가 길어지면 초기 정보가 사라지는 문제.
2. **Gate:** 시그모이드를 사용해 정보의 통과 여부(0 1)를 결정하는 밸브.
3. **GRU:** Update/Reset 게이트를 사용해 기억을 보존하거나 갱신한다.
4. **Key:** $\Gamma_u = 0$ 일 때 과거 기억이 손실 없이 그대로 전달된다.